

Kubernetes Production Guide for Senior Backend Interviews

1) Kubernetes Architecture

Control plane:

- kube-apiserver: entry point.
- etcd: source of truth for cluster state.
- scheduler: assigns pods to nodes.
- controller-manager: reconciliation loops.

Worker node:

- kubelet.
- container runtime.
- kube-proxy (or eBPF-based alternatives).

Interview point:

- Kubernetes is declarative desired-state reconciliation, not imperative scripting.

2) Workload Primitives

2.1 Deployment

- Stateless app rollout and rollback.
- Rolling updates with maxUnavailable/maxSurge.

2.2 StatefulSet

- Stable identity and storage for stateful services.
- Ordered startup/shutdown behavior.

2.3 DaemonSet

- One pod per node.
- Used for log agents, node monitoring, security agents.

2.4 Job and CronJob

- Batch and scheduled workloads.

3) Scheduling and Resource Management

3.1 Requests and Limits

- Requests drive scheduling.
- Limits cap usage (CPU throttling, OOMKill risk).

Senior discussion:

- Wrong resource settings cause instability and noisy-neighbor effects.

3.2 Placement Controls

- Node selectors.

- Affinity/anti-affinity.
- Taints and tolerations.
- Topology spread constraints.

3.3 Priority and Preemption

- Important workloads can preempt lower-priority pods.

4) Networking Deep Dive

4.1 Service Types

- ClusterIP, NodePort, LoadBalancer, ExternalName.

4.2 Ingress

- HTTP routing, TLS termination, host/path rules.

4.3 Network Policies

- Micro-segmentation between pods/namespaces.
- Default deny + explicit allow model.

Interview detail:

- Service discovery via DNS.
- Pod IPs are ephemeral, Services provide stable virtual IP.

5) Storage and Stateful Workloads

- PV, PVC, StorageClass.
- Dynamic provisioning.
- Access modes and reclaim policies.

Operational caution:

- stateful apps need backup strategy beyond volume attachment.

6) Configuration and Secrets

- ConfigMap for non-sensitive config.
- Secret for sensitive material (prefer external secret managers).
- Immutable config patterns with versioned rollout.

7) Autoscaling

7.1 HPA

- Scales pod replicas based on CPU/memory/custom metrics.

7.2 VPA

- Adjusts pod resource requests/limits.

7.3 Cluster Autoscaler

- Adds/removes nodes based on unschedulable pods.

Interview caveat:

- Autoscaling without queue/load-shedding can still fail under spikes.

8) Deployment Safety Patterns

- Readiness probes gate traffic.
- Liveness probes detect unhealthy loops.
- Startup probes for slow boot.
- PodDisruptionBudget for voluntary disruptions.
- Canary and blue/green via ingress/service mesh.

9) Reliability and Incident Handling

Common incidents:

- CrashLoopBackOff due to config/env mismatch.
- OOMKilled due to memory limit too low.
- Pending pods due to insufficient resources/taints.
- DNS/network policy regressions.

Debug flow:

- `kubectl get pods/events`
- `kubectl describe pod`
- `kubectl logs --previous`
- check readiness/liveness behavior
- check node pressure and scheduler events

10) Observability in Kubernetes

Metrics:

- container CPU/memory.
- restart counts.
- request latency/error.
- queue lag and saturation.

Logging:

- structured logs with trace IDs.
- centralized aggregation.

Tracing:

- OpenTelemetry instrumentation and collector.

11) Security in Kubernetes

- RBAC least privilege.
- Namespace isolation and policy boundaries.
- Pod Security Standards.
- Image scanning and signed artifacts.
- Runtime controls (seccomp/AppArmor/capability drop).

12) Multi-Cluster and DR

Patterns:

- Active-passive with failover.
- Active-active with global traffic routing.
- Regional data strategy aligned with app consistency model.

Must mention:

- tested failover runbooks.
- config and secret synchronization strategy.

13) Example: Go API on Kubernetes

Architecture:

- Deployment with 3 replicas.
- HPA on CPU + custom RPS metric.
- Redis cache and Postgres backend.
- Kafka consumer deployment separate from API deployment.
- Ingress with canary support.

Key YAML snippet (resource and probes):

```
resources:
  requests:
    cpu: "250m"
    memory: "256Mi"
  limits:
    cpu: "1"
    memory: "1Gi"
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  periodSeconds: 5
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  periodSeconds: 10
```

14) Rapid-Fire Kubernetes Questions

1. Deployment vs StatefulSet?
 - Deployment for stateless workloads.
 - StatefulSet for stable identity/storage and ordered operations.
2. Why can liveness probes be dangerous?
 - Bad probe thresholds can cause restart storms under temporary dependency slowness.
3. How do you prevent noisy-neighbor issues?
 - Correct requests/limits, QoS awareness, and node pool isolation.
4. How would you roll out risky changes safely?
 - Canary with automated rollback on SLO regression.

5. How do you debug a pod stuck in Pending?

- Check events for scheduling constraints, resources, taints, PVC issues.

15) Story Prompts

- Migrated workloads to Kubernetes and reduced deployment risk.
- Handled production incident caused by bad readiness/liveness setup.
- Reduced infra cost with autoscaling and right-sizing.
- Improved MTTR via better dashboards and runbooks.

16) Kubernetes Snippets with Interview Explanations

16.1 Deployment + HPA + PDB

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: orders-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: orders-api
  template:
    metadata:
      labels:
        app: orders-api
    spec:
      containers:
        - name: app
          image: ghcr.io/example/orders-api:v1.2.0
          resources:
            requests:
              cpu: "250m"
              memory: "256Mi"
            limits:
              cpu: "1"
              memory: "1Gi"
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: orders-api
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: orders-api
  minReplicas: 3
  maxReplicas: 20
  metrics:
```

```

- type: Resource
  resource:
    name: cpu
    target:
      type: Utilization
      averageUtilization: 65
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: orders-api
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: orders-api

```

Interview explanation:

- HPA scales for demand spikes.
- PDB protects availability during node drains and upgrades.

16.2 NetworkPolicy Default Deny + Allow API to DB

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-ingress-policy
  namespace: prod
spec:
  podSelector:
    matchLabels:
      app: postgres
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: orders-api
      ports:
        - protocol: TCP
          port: 5432

```

Interview explanation:

- Enforces least-privileged east-west traffic model.

16.3 Fast Debugging Command Set

```
kubectl get pods -n prod
kubectl describe pod orders-api-abc123 -n prod
kubectl logs orders-api-abc123 -n prod --previous
kubectl get events -n prod --sort-by=.metadata.creationTimestamp
kubectl top pod -n prod
```

Interview explanation:

- This sequence quickly isolates crash, probe, scheduling, and pressure signals.

17) L4 Kubernetes Grilling Addendum

17.1 Requests/Limits and QoS in Incidents

- Missing requests breaks scheduler packing and autoscaler behavior.
- Over-tight limits create CPU throttling and OOM kills.
- Know QoS classes: Guaranteed, Burstable, BestEffort.

17.2 Rollout Safety Pattern

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 0
minReadySeconds: 10
progressDeadlineSeconds: 600
```

Interview explanation:

- Availability-first rollout plus explicit deadline for stuck deploy detection.

17.3 HPA + VPA + Cluster Autoscaler Reality

- HPA scales pods from metrics.
- VPA tunes requests/limits (careful with stateful workloads).
- Cluster Autoscaler scales nodes for unschedulable pods.
- Thrash happens when metric windows and cooldowns are misconfigured.

17.4 Security Hardening Baseline

- Pod Security Standards (restricted profile where possible).
- NetworkPolicies for east-west isolation.
- Runtime defaults: non-root, read-only rootfs, dropped capabilities.
- Secret scope minimization and rotation strategy.

17.5 Fast Incident Questions

1. Why is `CrashLoopBackOff` happening: app crash, bad probe, missing dependency, or OOM?
 2. Why are pods Pending: quotas, affinity, taints, insufficient resources?
 3. Why p95 regressed: CPU throttling, DNS, node pressure, or downstream saturation?
 4. What is rollback trigger and owner?
-

18) L5 Kubernetes Platform Patterns

18.1 Multi-Tenant Cluster Guardrails

- Namespace isolation + quota + limit ranges.
- Pod security admission standards.
- Network policy default deny and explicit allow paths.
- Per-tenant SLO/error-budget ownership.

18.2 Progressive Delivery at Platform Level

- Canary rollout with automated analysis on latency/error rates.
- Rollback trigger tied to burn-rate thresholds.
- Verify readiness probe quality before trusting rollout automation.

18.3 Controller and Operator Strategy

- Use built-in controllers first.
- Introduce custom operators only when repetitive day-2 operations justify lifecycle complexity.

19) Kubernetes Failure Mode Drills

19.1 Pending Pods Under Load

Root-cause branches:

- unschedulable resources,
- node selectors/affinity conflicts,
- quota restrictions,
- autoscaler lag.

19.2 CrashLoopBackOff with Healthy Infra

Root-cause branches:

- startup contract mismatch,
- missing secret/config,
- aggressive probes,
- dependency boot order assumptions.

19.3 Traffic Blackhole During Rollout

Root-cause branches:

- readiness probe false positives,
- service selector mismatch,
- ingress rule conflict,
- network policy regression.

20) L5 Grilling Questions (Kubernetes)

1. How do you limit blast radius from noisy-neighbor workloads?
2. Why can HPA and cluster autoscaler fight each other?
3. How do you validate network policy changes safely before production?
4. What does a safe control-plane upgrade path look like at scale?
5. How do you define platform SLOs separate from app SLOs?

21) 60-Minute Kubernetes Revision Sprint

0-20 min:

- Scheduling, QoS, requests/limits, autoscaler interactions.

20-40 min:

- Rollout safety, probes, policies, and security baseline.

40-60 min:

- Incident triage narratives for CrashLoop/Pending/latency regression.
-